Dashboard Frontend stack

Critère	Poids	React	Angular	Vue.js	React + Next.js
Facilité d'apprentissage	Moyennement important	2 - React demande de maîtriser JavaScript, JSX et la logique des composants.	3 - Angular introduit de nombreux concepts comme TypeScript, les services et l'injection de dépendances.	1 - Vue est accessible grâce à une syntaxe proche du HTML, CSS et JavaScript.	2 - Next.js ajoute aux bases de React des conventions de routage, rendu et organisation du projet.
Facilité d'utilisation	Très important	1 - React permet de construire facilement une interface modulaire grâce aux composants réutilisables.	2 - Angular fournit beaucoup d'outils intégrés mais impose davantage de conventions.	1 - Vue permet de créer rapidement des composants lisibles et réutilisables.	1 - Next.js conserve la simplicité des composants React tout en structurant davantage l'application.
Routage frontend	Moyennement important	2 - React nécessite généralement une bibliothèque supplémentaire comme React Router.	1 - Angular intègre nativement un routeur complet.	2 - Vue utilise généralement Vue Router, qui doit être ajouté et configuré.	1 - Next.js propose un routage intégré basé directement sur l'arborescence des fichiers.
Actualisation	Moyennement important	1 - React met à jour dynamiquement l'interface lorsque l'état des composants change.	1 - Angular dispose d'un système réactif intégré pour mettre à jour l'interface.	1 - Vue met automatiquement à jour l'affichage lorsque ses données changent.	1 - Next.js bénéficie des mécanismes réactifs de React pour actualiser l'interface sans rechargement complet.
Dockerisation	Important	1 - Une application React statique peut être facilement construite puis servie avec Nginx.	1 - Angular peut être compilé puis servi simplement avec Nginx dans Docker.	1 - Vue peut être compilé puis distribué avec un serveur web léger comme Nginx.	1 - Next.js se dockerise facilement, soit en statique, soit avec un serveur Node.js selon les besoins.
Facilité de maintenance	Peu important	2 - React facilite la réutilisation mais laisse une grande liberté dans l'organisation du projet.	1 - Angular impose une structure claire qui facilite la maintenance des gros projets.	2 - Vue reste simple à maintenir mais nécessite des choix d'architecture lorsque le projet grandit.	2 - Next.js structure bien les pages et le routage mais demande une organisation rigoureuse du projet.
TOTAL		**9**	**10**	**8**	**8**

1 = favorable, 2 = intermédiaire, 3 = plus exigeant.

Nous avons choisi React avec Next.js pour le frontend car cette stack offre un bon compromis entre simplicité de développement, composants réutilisables et routage intégré. Elle est particulièrement adaptée à notre dashboard, qui repose sur plusieurs pages et de nombreux widgets interactifs, tout en restant facilement dockerisable et maintenable.

Une autre stack que nous aurions pu choisir est Vue.js. Elle obtient également un très bon score dans notre benchmark grâce à sa facilité d'apprentissage, sa simplicité d'utilisation et son système de réactivité intégré. Vue.js aurait été adapté à notre dashboard grâce à ses composants réutilisables et à son organisation simple : chaque composant Vue peut regrouper son affichage, sa logique et ses styles dans un même fichier .vue, ce qui rend les composants faciles à lire et à organiser.